



安徽大学

《机器人语音信息处理实验》

实验报告

实验名称: 基于动态时间规整的语音识别实验

学 院: 人工智能学院

专 业: 机器人工程

学 号: WA2224013

姓 名: 郭义月

授课老师: 杨普海

目录

【实验目的】	3
【实验原理】	3
1.模板匹配法语音识别系统构成.....	3
2.提取美尔频率倒谱系数.....	3
3. MFCC 系数的计算	4
4. 动态时间规整 (DTW).....	5
【实验预习】	7
1.基础知识的学习.....	7
2.实验环境.....	7
3.预习思考.....	7
4.可能遇到的问题.....	8
【实验内容】	8
实验 1：提取 mat 文件.....	8
实验 2：基于动态时间规整（DTW）的孤立字语音识别.....	9
实验 3：比较不同训练数据量对识别结果的影响.....	11
3.1 语调库中三条录音.....	11
3.2 语调库中二条录音.....	13
3.3 语调库中一条录音.....	15
实验 4：将欧式距离换成其他距离.....	15
4.1 曼哈顿距离.....	15
4.2 切比雪夫距离.....	16
4.3 明可夫斯基距离.....	17
4.4 余弦相似度.....	18
【小结与讨论】	19
1.学习了模板匹配和语音识别的理论知识.....	19
2. 实现了 DTW 孤立字的识别.....	19
3. 调整语调库中的数据量比较准确率.....	19
4. 不同的计算距离的公式会影响准确性.....	19
5. 实验思考.....	19

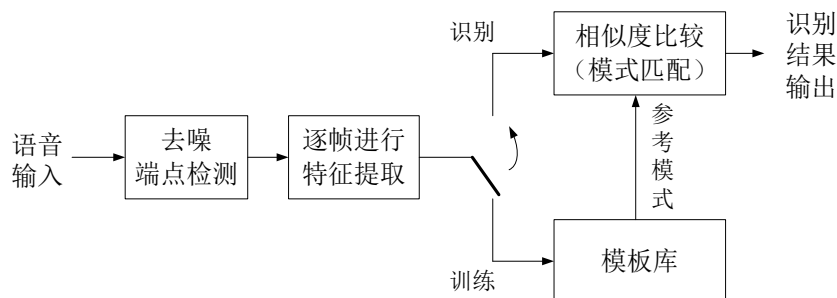
【实验目的】

- 1) 掌握语音识别的模板匹配法的原理和过程。
- 2) 掌握动态时间规整(DTW)技术和美尔频率倒谱系数特征提取技术。
- 3) 应用 MATLAB 实现基于 DTW 的 10 个阿拉伯数字的识别。

【实验原理】

1.模板匹配法语音识别系统构成

在训练阶段，用户将词汇表中的每一个词依次说一遍，并且将其特征矢量时间序列作为模板 (Template) 存入模板库；在识别阶段，将输入语音的特征矢量时间序列依次与模板库中的每个模板进行相似度比较，将相似度最高者作为识别结果输出。

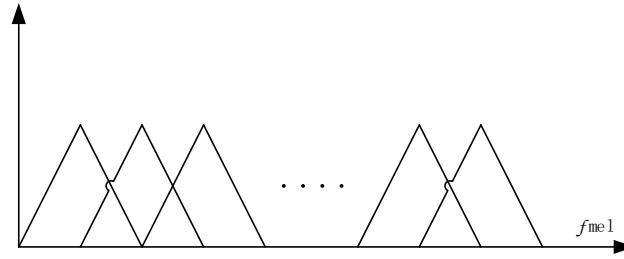


在特征提取阶段，本实验选用美尔频率倒谱系数(MFCC)作为识别特征；在识别阶段，实验选用动态时间规整(DTW)技术来进行模式匹配。下面将简单介绍这两个技术的原理和实现过程。

2.提取美尔频率倒谱系数

本实验采用美尔频率倒谱系数 (Mel-Frequency Cepstral Coefficients, MFCC)及其一阶和二阶差分作为特征参数。美尔频率倒谱系数不同于线性预测系数，它是将人耳的听觉特性和语音产生相结合的一种特征参数。

与普通倒谱分析不同，美尔倒谱系数着眼于分析人耳的听觉特性。人耳所听到的声音的高低与声音的频率并不呈线性正比关系，而用美尔频率尺度则更符合人耳的听觉特性。所谓美尔频率尺度，大体上对应于实际频率的对数分布关系。由前可知，人耳基底膜可分为许多小的部分，每一部分对应于一个频率群，对应于同一频率群的声音，在大脑中是叠加在一起进行评价的。这些频率群称为临界频带，其带宽随着频率的变化而变化，并与美尔频率的增长一致，在 1000 Hz 以下，大致呈线性分布，带宽为 100 Hz 左右；在 1000 Hz 以上呈对数增长。



类似于临界频带的划分，语音频率可以划分成一系列三角形的滤波器序列，即美尔滤波器组。设划分的带通滤波器为 $H_m(k)$, $0 \leq m \leq M$, M 为滤波器的个数。每个滤波器具有三角形滤波特性，其中心频率为 $f(m)$ ，在美尔频率范围内，这些滤波器是等带宽的。每个带通滤波器的传递函数为

$$H_m(k) = \begin{cases} 0 & k < f(m-1) \\ \frac{k-f(m-1)}{f(m)-f(m-1)} & f(m-1) \leq k \leq f(m) \\ \frac{f(m+1)-k}{f(m+1)-f(m)} & f(m) \leq k \leq f(m+1) \\ 0 & k > f(m+1) \end{cases}$$

其中， $\sum_{m=0}^{M-1} H_m(k) = 1$ 。

美尔滤波器的中心频率 $f(m)$ 定义为

$$f(m) = \frac{N}{f_s} F_{Mel}^{-1}(F_{Mel}(f_l) + m \frac{F_{Mel}(f_h) - F_{Mel}(f_l)}{M+1})$$

其中 f_h 和 f_l 分别为滤波器组的最高频率和最低频率， f_s 为采样频率，单位为 Hz。 M 是滤波器组的数目， N 为 FFT 变换的点数， $F_{Mel}^{-1}(b) = 700(e^{\frac{b}{1125}} - 1)$ 。

3. MFCC 系数的计算

MFCC 特征参数提取原理框图：



(1) 预处理

预处理包括预加重、分帧、加窗。

预加重：预加重的目的是为了补偿高频分量的损失，提升高频分量。预加重的滤波器常设为

$$H(z) = 1 - az^{-1}$$

式中， a 为一个常数，如 0.97。

分帧处理：通常语音信号是准稳态的信号，但是将它分为较短的帧后，每帧信号可以当作稳态信号，从而用处理稳态信号的方法来处理。同时，为了使一帧与

另一帧之间的参数能较平稳地过渡，因此相邻两帧之间会进行部分重叠处理。

加窗：加窗的目的是减少频域中的泄漏，将每一帧语音乘以汉明窗或海宁窗。语音信号 $x(n)$ 经预处理后为 $x_i(m)$ ，其中下标 i 表示分帧后的第 i 帧。

(2) 快速傅里叶变换

对每一帧信号进行 FFT 变换，从时域数据转变为频域数据

$$X(i, k) = FFT[X; (m)]$$

(3) 计算谱线能量

计算每一帧 FFT 后的数据谱线能量：

$$E(i, k) = [X_i(k)]^2$$

(4) 计算美尔滤波器能量

把求出的每帧谱线能量谱通过美尔滤波器，并计算在该美尔滤波器中的能量。在频域中相当于把每帧的能量谱 $E(i, k)$ (其中 i 表示第 i 帧， k 表示频域中的第 k 条谱线) 与美尔滤波器的频域响应 $H_m(k)$ 相乘并相加

$$S(i, m) = \sum_{k=0}^{N-1} E(i, k) H_m(k), 0 \leq m < M$$

(5) 计算 DCT 倒谱

序列 $x(n)$ 的 FFT 倒谱 $\hat{x}(n)$ 为

$$\hat{x}(n) = FT^{-1}[\hat{X}(k)]$$

式中， $\hat{X}(k) = \ln\{FT[x(n)]\} = \ln\{X(k)\}$, FT 和 FT^{-1} 表示傅里叶变换和傅里叶逆变换。序列 $x(n)$ 的 DCT 为

$$X(k) = \sqrt{\frac{2}{N}} \sum_{n=0}^{N-1} C(k) x(n) \cos\left[\frac{\pi(2n+1)k}{2N}\right], k = 0, 1, \dots, N-1$$

式中，参数 N 是序列 $x(n)$ 的长度； $C(k)$ 是正交因子，可表示为

$$C(k) = \begin{cases} \sqrt{2}/2 & k = 0 \\ 1 & k = 1, 2, \dots, N-1 \end{cases}$$

在上式中求取 FFT 的倒谱是把 $X(k)$ 取对数后计算 FFT 的逆变换。而这里求 DCT 的倒谱和求 FFT 的倒谱相类似，把美尔滤波器的能量取对数后计算 DCT:

$$mfcc(i, n) = \sqrt{\frac{2}{M}} \sum_{m=0}^{M-1} \ln[S(i, m)] \cos\left[\frac{\pi n(2m-1)}{2M}\right]$$

式中， $S(i, m)$ 是由式(10-6)求出的美尔滤波器能量； m 是指第 m 个美尔滤波器(共有 M 个)； i 是指第 i 帧； n 是 DCT 后的谱线。

4. 动态时间规整 (DTW)

在识别阶段的模式匹配中，不能简单地将输入模板和词库中的模板相比较来实现识别。因为语音信号具有相当大的随机性，这些差异不仅包括音强的大小、频谱的偏移，更重要的是发音持续时间不可能完全相同，而词库中的模板不可能随着

输入模板持续时间的变换而进行伸缩，所以时间规整是必不可少的。动态时间规整 (DTW) 是把时间规整和距离测度计算结合起来的一种非线性规整技术，它是模板匹配的方法。

假设词库中的某一参考模板的特征矢量列为 $a_1, \dots, a_m, \dots, a_M$, 另一方面, 输入语音的特征矢量序列为 $b_1, \dots, b_n, \dots, b_N, M \neq N$, 那么动态时间规整就是要找到时间规整函数 $m = T(n)$, 它把输入模板的时间轴 n 非线性的映射到参考模板的时间轴 m , 并且该函数满足下式:

$$D = \min_{T(n)} \sum_{n=1}^N d[n, T(n)]$$

式中, $d[n, T(n)]$ 表示两帧矢量之间的距离; D 是最佳时间路径下两个模板的距离测度。

本实验中距离测度采用欧氏距离: $d(x, y) = \frac{1}{k} \sqrt{\sum_{i=1}^k (x_i - y_i)^2}$ 。

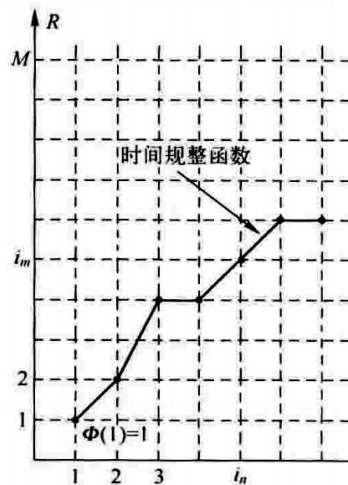
动态时间规整是一个典型的最优化问题, 它用满足一定条件的的时间规整函数 $T(n)$ 描述输入模板和参考模板之间的时间对应关系, 求解两模板匹配时累计距离最小所对应的规整函数。动态时间规整 (DTW) 一般采用动态规划 (DP) 算法实现。为了使计算更加简化, 它把一个 N 阶段的决策过程转化为 N 个简单段的决策过程, 也就是为 N 个子问题逐一做出决策。根据语音信号的性质, 时间规整函数必须满足以下约束条件:

1) 边界条件: $T(1) = 1, T(N) = M$

2) 单调性: $T(n+1) - T(n) \geq 0$

3) 连续性限制: 有些特殊的音素有时会对正确地识别起到很大的帮助, 某个音素的差异很可能就是区分不同的发声单元的依据, 为了保证信息损失最小, 规整函数一般规定不允许跳过任何一点。即

$$\Phi(i_n + 1) - \Phi(i_n) \leq 1$$



把测试模板的各个帧号 $n = 1 \sim N$ 在一个二维直角坐标系中的横轴上标出，把参考模板的各帧 $m = 1 \sim M$ 在纵轴上标出，通过这些表示帧号的整数坐标画出一些纵横线即可形成一个网格，网格中的每一个交叉点表示测试模式中某一帧与训练模式中某一帧的交汇。DTW 算法分两步进行，一是计算两个模式各帧之间的距离，即求出帧匹配距离矩阵，二是在帧匹配距离矩阵中找出一条最佳路径。路径搜索的过程可以描述如下：

搜索从 $(1,1)$ 点出发，对于局部路径约束，点 (i_n, i_m) 可达到的前一个格点只可能是 (i_{n-1}, i_m) 、 (i_{n-1}, i_{m-1}) 和 (i_{n-1}, i_{m-2}) 。那么 (i_n, i_m) 一定选择这三个距离中的最小者所对应的点作为其前续格点，这时此路径的累积距离为

$$D(i_n, i_m) = d(T(i_n), R(i_m)) + \min\{D(i_{n-1}, i_m), D(i_{n-1}, i_{m-1}), D(i_{n-1}, i_{m-2})\}$$

这样从 $(1,1)$ 点 $(D(1,1) = 0)$ 出发搜索，反复递推，直到 (N, M) 就可以得到最优路径，而且 $D(N, M)$ 就是最佳匹配路径所对应的匹配距离。在进行语音识别时，将测试模板与所有参考模板进行匹配，得到的最小匹配距离 $D_{\min}(N, M)$ 所对应语音即为识别结果。

【实验预习】

1.基础知识的学习

语音识别基础：了解语音识别的核心流程，包括信号预处理、特征提取、模式匹配。模板匹配法作为传统方法，通过训练阶段存储特征模板，识别阶段计算输入语音与模板的相似度实现分类。

MFCC 特征提取：美尔频率倒谱系数（MFCC）结合人耳听觉特性，将线性频率转换为美尔频率，通过滤波器组、对数运算和 DCT 变换提取特征，能有效表征语音的频谱包络。

DTW 动态时间规整：针对语音信号时长变化问题，DTW 通过非线性时间映射对齐不同长度的特征序列，基于动态规划求解最小累积距离，适用于孤立词识别。

2.实验环境

Matlab2021b，笃行南楼 A104

3.预习思考

在语音识别实验中，选择 MFCC 作为特征参数是因其结合人耳听觉特性，通过美尔滤波器组处理更贴近感知，而线性预测系数侧重信号生成模型；DTW 通过动态规划和非

线性时间映射解决时长不一致问题，边界条件和单调性约束确保时序对应；不同距离度量中，余弦距离因关注频谱方向相似性更适配语音特征，欧式、曼哈顿距离侧重数值差异，明可夫斯基距离受参数影响大，致识别准确率不同。

4.可能遇到的问题

1.代码实现：MFCC 特征提取函数（mfccf）的参数配置（如滤波器数量、帧长）可能影响特征质量；DTW 算法的动态规划路径搜索逻辑易出现索引错误。

2.数据处理：端点检测（my_vad）可能受环境噪声干扰，导致有效语音段提取不完整；训练数据量不足时，模型泛化能力差。

3.结果分析：不同距离度量的识别准确率差异可能需结合语音特性（如频谱分布、时序动态）深入分析。

【实验内容】

实验 1：提取 mat 文件

本实验的语音来自三个人，每人说一遍 0~9 这十个数字，记录下相应的语音，保存在目录“SpeechData”中。以第一个人的语音“1”为例，首先，调用 [speechIn1,FS1]=audioread(q)读入语音“1”，保存在 speechIn1 中，并获得采样率 FS1。

```
%设置模板
ncoeff=12;                %mfcc 系数的个数
fMatrix1 = cell(1,10);
fMatrix2 = cell(1,10);
fMatrix3 = cell(1,10);
```

对 speechIn1 进行预处理，首先调用 my_vad 函数进行端点检测，提取出有效语音段，然后调用 mfccf 函数对有效语音段提取 MFCC 特征。

```
for i = 1:10
    q = ['./SpeechData/p1/' num2str(i-1) '.wav'];
    [speechIn1,FS1] = audioread(q);
    speechIn1 = my_vad(speechIn1);
    fMatrix1(1,i) = {mfccf(ncoeff,speechIn1,FS1)};

end

for j = 1:10
```



```

    q = ['./SpeechData/p2/' num2str(j-1) '.wav'];
    [speechIn2,FS2] = audioread(q);
    speechIn2 = my_vad(speechIn2);
    fMatrix2(1,j) = {mfccf(ncoeff,speechIn2,FS2)};
end

for k = 1:10
    q = ['./SpeechData/p3/' num2str(k-1) '.wav'];
    [speechIn3,FS3] = audioread(q);
    speechIn3 = my_vad(speechIn3);
    fMatrix3(1,k) = {mfccf(ncoeff,speechIn3,FS3)};
end

```

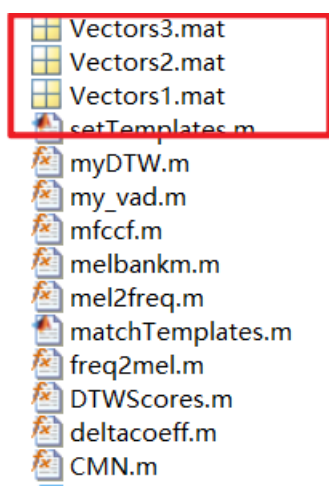
以同样的方式处理其他训练语音，把第一/二/三个说话人的十个数字 0~9 语音分别保存在 s1/s2/s3 结构体中，并最终保存在 Vectors1.mat /Vectors2.mat /Vectors3.mat 文件中。

```

%将数据保存为 mat 文件
fields =
{'zero', 'One', 'Two', 'Three', 'Four', 'Five', 'Six', 'Seven', 'Eight', 'nine'};
s1 = cell2struct(fMatrix1, fields, 2);          %fields 项作为行
save Vectors1.mat -struct s1;
s2 = cell2struct(fMatrix2, fields, 2);
save Vectors2.mat -struct s2;
s3 = cell2struct(fMatrix3, fields, 2);
save Vectors3.mat -struct s3;

```

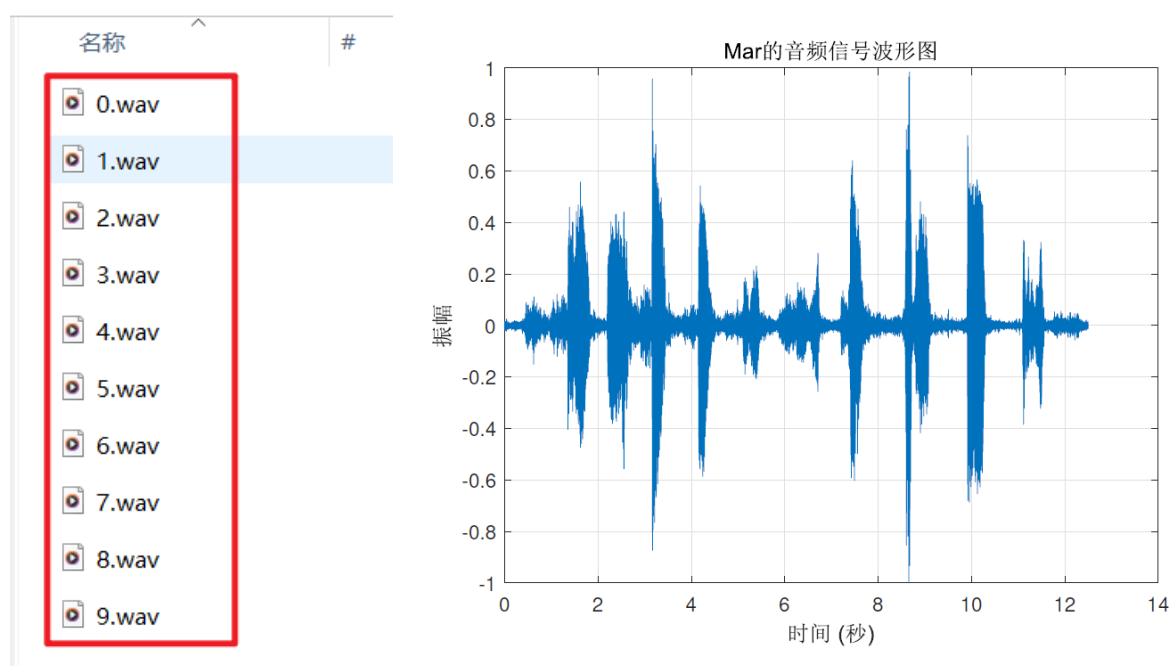
实验结果：



fMatrix1(1,1)								
提取的MFCC特征								
1	2	3	4	5	6	7	8	
-158.3146	-198.9447	-233.3617	-274.9841	-296.8303	-269.0537	-269.7727	-209.5844	-
-153.3818	-183.0888	-211.5159	-258.6347	-300.2163	-280.8040	-273.8832	-213.6285	-
-150.3082	-202.7235	-226.0814	-265.5793	-287.8808	-274.0238	-307.3259	-213.7499	-
-146.0321	-182.5857	-220.9428	-271.4350	-282.2659	-264.9566	-300.8637	-226.8098	-
-142.5311	-194.0163	-223.6587	-258.9905	-287.9748	-278.8916	-286.6984	-209.7821	-
-158.2428	-200.7390	-231.4145	-264.6469	-283.6141	-263.6440	-269.0368	-212.9570	-
-147.3126	-200.4310	-224.1287	-275.3461	-281.5762	-240.1115	-285.7229	-214.8510	-
-145.0849	-200.7175	-215.6189	-263.1584	-289.6812	-262.8044	-295.8852	-213.0429	-
-149.9235	-236.2335	-227.9145	-235.2495	-274.1044	-270.3728	-310.3210	-212.5787	-
-145.7953	-197.9767	-211.9789	-257.8528	-308.2853	-283.4520	-285.8378	-223.9778	-
-146.5188	-212.8791	-246.5904	-287.4642	-309.0007	-271.1023	-274.6563	-209.2906	-
-154.3912	-191.9225	-219.8177	-266.3666	-284.2011	-270.1350	-305.6139	-214.5671	-
-150.2691	-198.2968	-211.8758	-253.5714	-281.0244	-253.7031	-270.7269	-211.1649	-
-146.1176	-197.6685	-218.1831	-247.8110	-270.4032	-263.4407	-297.8908	-226.1834	-
-140.9915	-204.4188	-230.1859	-257.0894	-285.0098	-270.3861	-281.2332	-221.1183	-
-145.1312	-174.0999	-203.9359	-252.8154	-291.0605	-278.4859	-283.1406	-240.0179	-
-150.5948	-186.3457	-200.0656	-251.2094	-300.8381	-285.4181	-294.7232	-223.7209	-
-152.9328	-183.4691	-199.6538	-240.2416	-261.2246	-251.5829	-285.8690	-225.1899	-
-138.3190	-184.4609	-211.4170	-255.2312	-294.0159	-290.0053	-299.5663	-232.1789	-
-123.1406	-170.2414	-191.4114	-236.8151	-285.3015	-288.9401	-316.9077	-258.3315	-

实验 2：基于动态时间规整（DTW）的孤立字语音识别

由于在实验室不适合录音，所以可以把提前录制好的文件读入，提前录制了中文的录音，已经保存到文件夹中：



读取的代码为：

```
[speechIn,FS1] = audioread('./Mar/Marker 3.wav');
```

对 speechIn 进行预处理，包括端点检测和 MFCC 特征提取，这一过程和训练中完全相同，得到(N×P)矩阵，其中N为帧数，P为维度，接着调用CMN函数，对每一维度分量进行标准化，结果保存在rMatrix中。

```
speechIn = my_vad(speechIn); %端点检测
rMatrix1 = mfccf(ncoeff,speechIn,fs); %采用 MFCC 系数作为特征矢量
rMatrix = CMN(rMatrix1); %归一化处理
```

调用 DTWScores 函数，将当前待识别语音的 rMatrix 与训练过程中保存的 s1、s2、s3 中的 0~9 各数字进行匹配，此函数中调用 myDTW 函数实现基于 DTW 的匹配，返回 rMatrix 和每个说话人说的每个数字的匹配分值，保存在 Sco((3*10) 矢量)。

```
Sco = DTWScores(rMatrix,N); %计算 DTW 值
```

将 Sco 从大到小进行排序，首先得到每个模板匹配的两个最低分值对应的数字序号 (有 2*3=6 个序号),而后返回出现频率最高的序号作为判决结果，从而完成识别过程。

```
[SortedScores,EIndex] = sort(Sco,2); %按行递增排序，并返回对应的原始次序
```



```

speechIn = my_vad(speechIn); %端点检测
rMatrix1 = mfccf(ncoeff,speechIn,fs); %采用 MFCC 系数作为特征

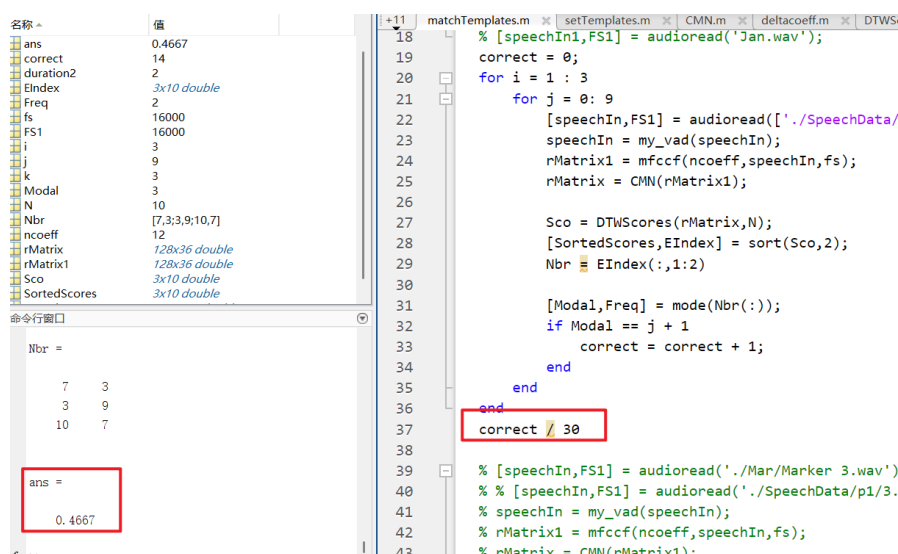
%归一化处理
rMatrix = CMN(rMatrix1);

%计算 DTW 值
[SortedScores,EIndex] = sort(Sco,2); %按行递增排序，并返回对
应的原始次序
Nbr = EIndex(:,1:2) %得到每个模板匹配的 2 个最
低值对应的次序

[Modal,Freq] = mode(Nbr(:)); %返回出现频率最高的数
Modal 及其出现频率 Freq
if Modal == j + 1
    correct = correct + 1;
end
end
end
correct / 30

```

正确率为:



可见，即使在训练集上，正确率只有 0.47，如果在测试集上：

修改代码如下：

```

correct = 0;
for j = 0: 9
    [speechIn,FS1] = audioread(['./Mar/Marker ' num2str(j) '.wav']);
    speechIn = my_vad(speechIn); %端点检测
    rMatrix1 = mfccf(ncoeff,speechIn,fs); %采用 MFCC 系数作为特征矢量
    rMatrix = CMN(rMatrix1); %归一化处理

```

```

Sco = DTWScores(rMatrix,N);
[SortedScores,EIndex] = sort(Sco,2);
原始次序
Nbr = EIndex(:,1:2)
对应的次序

[Modal,Freq] = mode(Nbr(:));
及其出现频率 Freq
if Modal == j + 1
    correct = correct + 1;
end
end
correct / 10

```

%计算 DTW 值
%按行递增排序, 并返回对应的
%得到每个模板匹配的 2 个最低值
%返回出现频率最高的数 Modal

结果为:

The image shows a MATLAB interface with two main panels. The left panel, titled '名称' (Name) and '值' (Value), lists variables and their types. The right panel shows the script 'matchTemplates.m' with line numbers 24 to 48. The script contains MATLAB code for template matching, including reading audio, computing DTW scores, and counting correct matches. The final result 'correct / 10' is highlighted with a red box, showing the value 0.2000.

名称	值
ans	0.2000
correct	2
duration2	2
EIndex	3x10 double
Freq	2
fs	16000
FS1	48000
j	9
k	3
Modal	4
N	10
Nbr	[5,1;4,5;2,4]
ncoeff	12
rMatrix	459x36 double
rMatrix1	459x36 double
Sco	3x10 double
SortedScores	3x10 double
speechIn	73601x1 double

```

24 % correct = correct + 1;
25 % end
26 % end
27 % end
28 % correct / 30
29
30 correct = 0;
31 for j = 0: 9
32     [speechIn,FS1] = audioread(['./Mar/Marker ' num2st
33     speechIn = my_vad(speechIn);
34     rMatrix1 = mfccf(ncoeff,speechIn,fs);
35     rMatrix = CMN(rMatrix1);
36
37     Sco = DTWScores(rMatrix,N);
38     [SortedScores,EIndex] = sort(Sco,2);
39     Nbr = EIndex(:,1:2);
40
41     [Modal,Freq] = mode(Nbr(:));
42     if Modal == j + 1
43         correct = correct + 1;
44     end
45 end
46 correct / 10
47
48

```

可见, 在一条没有训练过的语音上面, 正确率只有 0.2, 说明 DTW 并没有很好的泛化性, 且自身的训练结果并不理想

3.2 语调库中二条录音

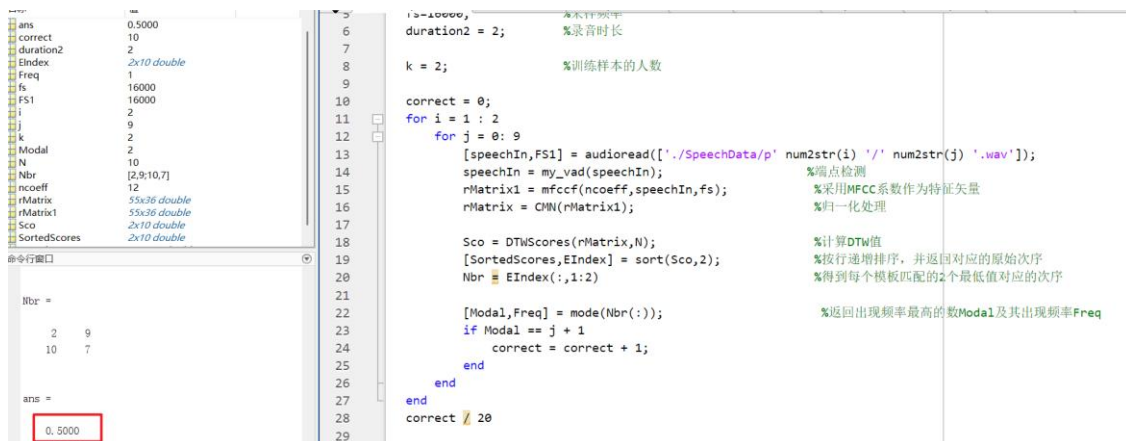
只需要调整函数的输出值, 就可以设置为两条语音:

```

+11 matchTemplates.m DTWScores.m setTemplates.m CMN.m deltacoeff.m freq2mel.m
19
20
21 %计算DTW
22 for i = 1:N
23     fMatrix1 = fMatrixall1{i,1};
24     fMatrix1 = CMN(fMatrix1);
25     Scores1(i) = myDTW(fMatrix1,rMatrix);
26 end
27
28 for j = 1:N
29     fMatrix2 = fMatrixall2{j,1};
30     fMatrix2 = CMN(fMatrix2);
31     Scores2(j) = myDTW(fMatrix2,rMatrix);
32 end
33
34 for k= 1:N
35     fMatrix3 = fMatrixall3{k,1};
36     fMatrix3 = CMN(fMatrix3);
37     Scores3(k) = myDTW(fMatrix3,rMatrix);
38 end
39
40 AllScores = [Scores1;Scores2];
41
42 % AllScores = [Scores1;Scores2;Scores3];

```

在初始的训练集上:



```

1 % 16000, %采样频率
2 duration2 = 2; %录音时长
3
4 k = 2; %训练样本的人数
5
6 correct = 0;
7 for i = 1 : 2
8     for j = 0: 9
9         [speechIn,FS1] = audioread(['./SpeechData/p' num2str(i) '/' num2str(j) '.wav']);
10        speechIn = my_vad(speechIn); %端点检测
11        rMatrix1 = mfccf(ncoeff,speechIn,fs); %采用MFCC系数作为特征矢量
12        rMatrix = CMN(rMatrix1); %归一化处理
13
14        Sco = DTWScores(rMatrix,N); %计算DTW值
15        [SortedScores,EIndex] = sort(Sco,2); %按行递增排序，并返回对应的原始次序
16        Nbr = EIndex(:,1:2); %得到每个模板匹配的2个最低值对应的次序
17
18        [Modal,Freq] = mode(Nbr(:)); %返回出现频率最高的数Modal及其出现频率Freq
19        if Modal == j + 1
20            correct = correct + 1;
21        end
22    end
23 end
24 correct / 20
25
26
27
28
29

```

命令窗口:

```

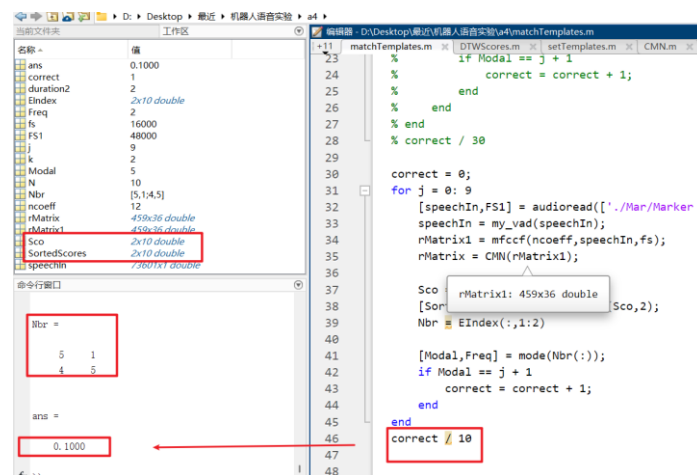
Nbr =
     2     9
    10     7

ans =
    0.5000

```

正确率为 0.5

在测试集上结果为:



```

+11 matchTemplates.m DTWScores.m setTemplates.m CMN.m
23 %
24 if Modal == j + 1
25     correct = correct + 1;
26 end
27 % end
28 % correct / 30
29
30 correct = 0;
31 for j = 0: 9
32     [speechIn,FS1] = audioread(['./Mar/Marker '
33     speechIn = my_vad(speechIn);
34     rMatrix1 = mfccf(ncoeff,speechIn,fs);
35     rMatrix = CMN(rMatrix1);
36
37     Sco = rMatrix1; % 459x36 double
38     [Sor = EIndex(:,1:2) Sco,2);
39
40     [Modal,Freq] = mode(Nbr(:));
41     if Modal == j + 1
42         correct = correct + 1;
43     end
44 end
45
46 correct / 10
47
48

```

命令窗口:

```

Nbr =
     5     1
     4     5

ans =
    0.1000

```

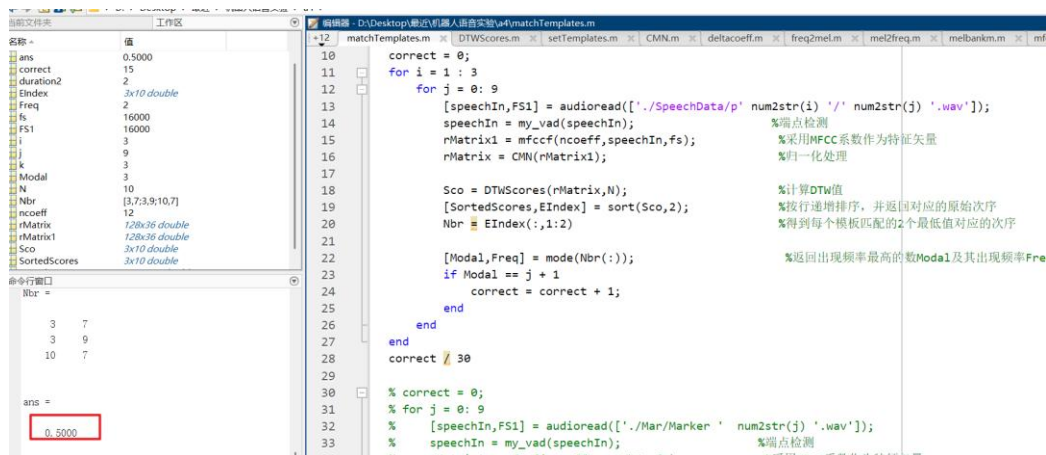
当语调库中有两条语音时，正确率只有 0.1


```

for n=1:r1
    for m=1:r2
        FR=abs(F(n,:)-R(m,:)); % 计算绝对值差
        distance(n,m)=sum(FR)/c1; % 曼哈顿距离
    end
end

```

实验结果:



修改为曼哈顿距离后，正确率由之前的 0.47 变成了 0.5

4.2 切比雪夫距离

切比雪夫距离是各坐标差的最大值，公式为：

$$d(n,m) = \frac{1}{c1} \max_i |F(n,i) - R(m,i)|$$

实验代码:

```

for n=1:r1
    for m=1:r2
        FR=abs(F(n,:)-R(m,:)); % 计算绝对值差
        distance(n,m)=max(FR)/c1; % 切比雪夫距离
    end
end

```

实验结果:

The screenshot shows a MATLAB environment. On the left, the 'Workspace' window lists variables: ans (0.4000), correct (12), duration2 (2), EIndex (3x10 double), Freq (2), fs (16000), FS1 (16000), i (3), j (9), k (3), Modal (3), N (10), Nbr ([7,3;3,9;10,7]), ncoeff (12), rMatrix (128x36 double), rMatrix1 (128x36 double), Sco (3x10 double), and SortedScores (3x10 double). Below this, the 'Command Window' shows the variable Nbr with its matrix value and the final result ans = 0.4000. On the right, the 'Editor' window contains MATLAB code for DTW-based speech recognition, including audio reading, MFCC calculation, DTW scoring, and a loop to find the best match.

```

13=16000; %采样频率
duration2 = 2; %录音时长

k = 3; %训练样本的人数

correct = 0;
for i = 1 : 3
    for j = 0: 9
        [speechIn,FS1] = audioread(['./SpeechData/p' num2str(i)
speechIn = my_vad(speechIn); %端点检测
rMatrix1 = mfccf(ncoeff,speechIn,fs); %采用MF
rMatrix = CMN(rMatrix1); %归一化

Sco = DTWScores(rMatrix,N); %计算DT
[SortedScores,EIndex] = sort(Sco,2); %按行递
Nbr = EIndex(:,1:2) %得到每

[Modal,Freq] = mode(Nbr(:)); %返回
if Modal == j + 1
    correct = correct + 1;
end
end
end
correct % 30
  
```

修改为切比雪夫距离后，正确率比欧式距离（0.47）少了 0.07

4.3 明可夫斯基距离

明可夫斯基距离是欧氏距离和曼哈顿距离的推广， p 是一个参数。当 $p=2$ 时，明可夫斯基距离就是欧氏距离；当 $p=1$ 时，就是曼哈顿距离。在本实验中，令 $p=3$

实验代码：

```

Sigma_inv = inv(cov([F;R])); % 计算协方差矩阵的逆
for n=1:r1
    for m=1:r2
        FR=F(n,:)-R(m,:);
        distance(n,m)=sqrt(FR*Sigma_inv*FR')/c1; % 马氏距离
    end
end
  
```

实验结果：

```

matchTemplates.m
1  % 录音时长
2  duration2 = 2;
3
4  % 训练样本的人数
5  k = 3;
6
7  correct = 0;
8  for i = 1 : 3
9      for j = 0: 9
10         [speechIn,FS1] = audioread(['./SpeechData/p' num2str(i) '/' num2str(j) '.wav'])
11         %端点检测
12         speechIn = my_vad(speechIn);
13         %采用MFCC系数作为特征矢量
14         rMatrix1 = mfccf(ncoeff,speechIn,fs);
15         %归一化处理
16         rMatrix = CMN(rMatrix1);
17
18         Sco = DTWScores(rMatrix,N);
19         %计算DTW值
20         [SortedScores,EIndex] = sort(Sco,2);
21         %按行递增排序，并返回对应的原始次序
22         Nbr = EIndex(:,1:2);
23         %得到每个模板匹配的2个最低值对应的次序
24
25         [Modal,Freq] = mode(Nbr(:));
26         %返回出现频率最高的数Modal及其出现频率F
27         if Modal == j + 1
28             correct = correct + 1;
29         end
30     end
31 end
32 correct
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100

```

ans =
0.1667

明可夫斯基距离的正确率只有 0.17

4.4 余弦相似度

余弦相似度衡量的是两个向量的方向相似性。

实验代码：

```

for n=1:r1
    for m=1:r2
        dot_product = dot(F(n,:), R(m,:)); % 点积
        norm_F = norm(F(n,:)); % F(n,:) 的范数
        norm_R = norm(R(m,:)); % R(m,:) 的范数
        distance(n,m) = 1 - dot_product / (norm_F * norm_R); % 余弦距离
    end
end

```

实验结果：

```

matchTemplates.m
1  % 录音时长
2  duration2 = 2;
3
4  % 训练样本的人数
5  k = 3;
6
7  correct = 0;
8  for i = 1 : 3
9      for j = 0: 9
10         [speechIn,FS1] = audioread(['./SpeechData/p' num2str(i) '/' num2str(j) '.wav']);
11         %端点检测
12         speechIn = my_vad(speechIn);
13         %采用MFCC系数作为特征矢量
14         rMatrix1 = mfccf(ncoeff,speechIn,fs);
15         %归一化处理
16         rMatrix = CMN(rMatrix1);
17
18         Sco = DTWScores(rMatrix,N);
19         %计算DTW值
20         [SortedScores,EIndex] = sort(Sco,2);
21         %按行递增排序，并返回对应的原始次序
22         Nbr = EIndex(:,1:2);
23         %得到每个模板匹配的2个最低值对应的次序
24
25         [Modal,Freq] = mode(Nbr(:));
26         %返回出现频率最高的数Modal及其出现频率F
27         if Modal == j + 1
28             correct = correct + 1;
29         end
30     end
31 end
32 correct
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100

```

ans =
0.6333

余弦相似度的准确率为 0.63

【小结与讨论】

1. 学习了模板匹配和语音识别的理论知识

通过实验掌握了模板匹配法的核心流程，明确了 MFCC 特征提取中预加重、分帧、加窗及美尔滤波器组的作用，理解了 DTW 算法通过动态规划解决时间对齐问题的原理。

2. 实现了 DTW 孤立字的识别

利用 MATLAB 完成了 10 个阿拉伯数字的孤立词识别系统，通过端点检测提取有效语音段，经 MFCC 特征提取和 CMN 归一化后，基于 DTW 算法与模板库匹配，成功验证了算法对特定语音的识别能力。

3. 调整语调库中的数据量比较准确率

数据量影响显著：当训练数据从 3 人（30 条）减少到 1 人（10 条）时，训练集准确率从 46.67% 降至 30%，测试集准确率从 20% 降至 10%，表明 DTW 对训练数据量依赖性强，泛化能力不足。这说明少量数据难以覆盖语音的个体差异（如发音习惯、语速），导致模板库代表性不足。

4. 不同的计算距离的公式会影响准确性

余弦相似度表现最佳：将欧式距离替换为余弦相似度后，训练集准确率提升至 63.33%，因其更关注特征向量的方向相似性，对语音频谱的整体结构匹配更鲁棒。

其他距离的局限性：曼哈顿距离（准确率 50%）和切比雪夫距离（准确率 40%）对局部特征差异更敏感，明可夫斯基距离（ $p=3$ 时准确率 16.67%）因参数设置复杂导致稳定性差。

5. 实验思考

传统 DTW 算法在数据量不足时识别率低，且仅适用于孤立词，无法处理连续语音。可考虑引入隐马尔可夫模型（HMM）或深度学习方法（如 CNN、RNN）提升泛化能力。